

Building a FuSa Documentation Automation Machine

Luke Renchik
Independent

29 April 2026

Abstract

Functional Safety evaluation and documentation are problems of data synthesis and data reasoning, which are prone to human error and require large ramp-up efforts. The goal of this tool is to synthesize, chunk, and categorically store all relevant information related to a product and leverage the use of LLM reasoning systems to assist in the evaluation and creation of the necessary documentation for these safety critical systems.



Contents

1	Part I — Foundations	2
1.1	Designing Data-Intensive Systems	2
1.1.1	Networking Protocols	2
1.1.2	Data-Intensive Application Fundamentals	4
1.1.3	Reliability	10
1.1.4	Scalability	11
1.1.5	Maintainability	14
2	Part 2: Data models and Query Languages	14
2.1	Data Models	14
2.1.1	Relational Model Versus Document Model	15
2.2	Graph-Like Data Models	18
2.2.1	Graph query languages	18
2.2.2	Property graphs	18
2.2.3	Triple stores and SPARQL	20
2.2.4	The Semantic Web & Summary	21

1 Part I — Foundations

Prerequisites and background knowledge required before implementation.

1.1 Designing Data-Intensive Systems

Started: 4/19/26

1.1.1 Networking Protocols

TCP (Transmission Control Protocol).

TCP operates at the transport layer. Its job is to take raw IP packets — which are inherently unreliable and unordered — and deliver a reliable, ordered byte stream between two endpoints.

Connection Lifecycle

1. Three-Way Handshake — establishes a connection before any data flows:

```
Client → SYN      (I want to connect, my seq# is X)
Server → SYN-ACK   (OK, my seq# is Y, I acknowledge X)
Client → ACK       (Acknowledged Y)
```

2. Data Transfer

- TCP numbers every byte with a sequence number
- Receiver sends ACKs confirming what was received
- Sender retransmits anything not ACKed within a timeout
- Reorders out-of-order segments using sequence numbers before handing to the application

TCP throttles transmission rate via:

- **Receive window** — how much the receiver can buffer
- **Congestion window** — how much the network can handle

3. Teardown — uses a four-way FIN/ACK exchange. Either side can initiate.

—

HTTP (Hypertext Transfer Protocol).

HTTP is the application protocol on top of TCP. It doesn't know or care about packets — it just writes bytes to a TCP socket and reads bytes back.

Request / Response Format

```
GET /api/users HTTP/1.1
Host: example.com
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 83
```

```
{"users": [...]}
```

HTTP Versions

Speed	Slower due to overhead	Faster, minimal overhead
Use Cases	HTTP, SSH, email, file transfer	Video streaming, gaming, DNS, VoIP, QUIC

UDP is essentially raw IP with port numbers — speed and control, but you implement any reliability you need yourself (which is exactly what QUIC does).

—

QUIC.

Transport protocol built on UDP, designed by Google and standardized in RFC 9000. HTTP/3 is HTTP/2's semantics running over QUIC instead of TCP.

- **Stream isolation** — packet loss only affects the one stream it belonged to
- **Connection IDs** — instead of IP + port tuples, so connections survive switching from WiFi to LTE
- **TLS 1.3 mandatory and baked in** — handshake is combined with the transport handshake, reducing round trips
- **0-RTT resumption** — for returning connections, data can flow before the handshake even completes

—

TLS 1.3.

Encryption layer beneath HTTPS. Version 1.3 (2018) was a significant cleanup over 1.2.

- **Faster handshake** — TLS 1.2 required 2 round trips before data could flow. TLS 1.3 does it in 1 round trip. With 0-RTT resumption, a returning client can send data with the very first packet.
- **Removed legacy cruft** — TLS 1.2 supported dozens of cipher suites including broken ones (RC4, MD5, SHA-1). TLS 1.3 cuts to 5 modern suites only.
- **Forward secrecy is mandatory** — if your private key is compromised in the future, recorded past sessions still can't be decrypted, because each session used a fresh ephemeral key.

—

1.1.2 Data-Intensive Application Fundamentals

DDIA — Chapter 1

The three pillars of any well-designed data-intensive system:

Pillar	Concern
Reliability	Tolerating hardware & software faults, human error
Scalability	Measuring load & performance, latency, percentiles, throughput
Maintainability	Operability, simplicity, & evolvability

Core functions of data-intensive applications:

Store Data	Databases
Speed Up Reads	Caches — remember the result of an expensive operation
Search & Filter	Search indexes — keyword search and filtering
Batch Processing	Periodically crunch large amounts of accumulated data

—

Redis (Remote Dictionary Server).

An open source, in-memory data structure store usable as a database, cache, message broker, and streaming engine. Keeps data primarily in RAM — typically handles millions of operations per second.

Common Use Cases

- **Caching layer** — sits in front of a slow database so frequent queries skip the DB entirely
- **Rate limiting** — track requests per IP using atomic increment operations
- **Pub/Sub** — lightweight async communication between services
- **Distributed locks** — SET NX allows services to coordinate shared resource access without race conditions

Race Conditions

A race condition occurs when program correctness depends on the relative timing of concurrent operations. Two or more threads “race” to read and modify shared state, with outcomes differing depending on who wins.

Thread A reads balance: \$150
Thread B reads balance: \$150 ← B reads before A has written
Thread A writes balance: \$50
Thread B writes balance: \$50 ← B overwrites A's result

Both withdrawals succeed, but the final balance is \$50 instead of \$-50. The window of vulnerability between a read and a subsequent write is called a **critical section**. Race conditions are non-deterministic — they may only surface under load, on specific hardware, or once in a million requests.

Atomic Operations

An operation is **atomic** if it executes as a single, indivisible unit — no observable intermediate state exists.

Thread A: reads \$150, writes \$50 (atomic)
Thread B: reads \$50 → rejects withdrawal (insufficient funds)

How atomicity is achieved:

- **Hardware (CAS)** — Compare-And-Swap: “only write if the current value is what I expect.” The foundation everything else builds on.

- **Mutexes/locks** — only one thread enters the critical section at a time
- **Semaphores** — generalized locks allowing N concurrent accessors
- **Atomic types** — e.g., `std::atomic<int>` in C++; hardware-backed, no explicit locking needed
- **Database ACID** — atomicity is the “A”: either all statements in a transaction commit or none do
- **Redis** — every command is atomic; Redis processes commands sequentially in a single thread

Wherever you have shared state and concurrency, reason carefully about what operations need to be atomic and over what scope.

—

Apache Kafka.

Core Abstraction: The Log

Kafka models message streams as an **append-only log**, exactly like a database’s write-ahead log (WAL).

No deletion on read	Messages persist for a configurable retention window
Offsets	Every message has a monotonically increasing integer index
Independent consumers	Each consumer tracks its own offset; can rewind and replay
Sequential writes	Append-only writes are extremely fast on disk

Terminology

- **Topic** — named stream of messages, analogous to a database table
- **Partition** — each topic split into N independent logs for parallelism

Topic: "orders"

```
Partition 0: [msg0] [msg3] [msg6] ...
Partition 1: [msg1] [msg4] [msg7] ...
Partition 2: [msg2] [msg5] [msg8] ...
```

- **Broker** — a single Kafka server; clusters run 3–N brokers for fault tolerance
- **Replication** — each partition has a leader + N replicas; writes go to leader, replicated before ACK

—

Application-Managed Caching & Memcached.

The application is explicitly responsible for what to cache, when to invalidate, and how to handle misses. The cache itself is a dumb key-value store with no knowledge of business logic.

Cache-Aside Pattern (Lazy Loading)

```
def get_user(user_id):
    cached = cache.get(f"user:{user_id}")
    if cached:
        return cached                                # cache hit
```

```

user = db.query("SELECT * FROM users WHERE id = ?", user_id)
cache.set(f"user:{user_id}", user, ttl=300)    # store for 5 min
return user                                    # cache miss, now populated

```

Cache Invalidation — one of the hardest problems in CS. Cache goes stale when:

- Underlying data changes and the cache is not updated
- Multiple services write to the same data through different paths
- A distributed system has inconsistent views across nodes

Strategies: **TTL expiration, explicit invalidation on write, versioned keys.**

Memcached — oldest and most widely deployed caching system. Focused entirely on speed and simplicity.

- Stores only strings (client serializes everything)
- No persistence, no replication, no server-side clustering logic
- Pure LRU eviction when memory is full
- **Distributed hashing** — client library maps keys to nodes via consistent hashing; deterministic and coordination-free
- **Slab allocation** — memory pre-divided into fixed-size chunk classes to avoid heap fragmentation

Memcached vs. Redis

Choose when...	Memcached	Redis
Data structures	Plain strings only	Lists, sets, hashes, streams, ...
Persistence	Not needed	Required (RDB / AOF)
Clustering	Client-managed simplicity	Server-managed built-in
Threading	True multi-threaded	Mostly single-threaded
Extra features	Not needed	Pub/Sub, scripting, transactions

Full Stack View

```

User Request
  ↓
Application Server
  ↓
Memcached / Redis ← cache-aside for DB query results, sessions
  ↓ (on miss)
PostgreSQL / MySQL ← source of truth
  ↓ (on writes)
Kafka ← emit change events for downstream consumers

```

Full-Text Search.

Elasticsearch or **Solr** — allow users to search data by keyword or filter in various ways.

A regular database index can answer “find the row where name = ‘John’.” A full-text search can find every document that **mentions** John, is **about** John, or is conceptually related to him. Instead of indexing rows by their primary key, you index every meaningful word in every document and build a map from words → documents. This is called an **inverted index**.

Word	→ Documents containing it (+ position, frequency)
"engine"	→ [doc3 (pos:4, tf:2), doc7 (pos:1, tf:1), doc12 (pos:9, tf:3)]
"failure"	→ [doc1, doc3, doc7, doc19]
"safety"	→ [doc1, doc2, doc7, doc15]
"automotive"	→ [doc2, doc7, doc12]

Relevance Scoring

- **TF-IDF** — Term Frequency (how often a term appears in the document) × Inverse Document Frequency (how rare the term is across the corpus). Common terms like “the” score near zero; rare terms like “FMEDA” score high.
- **BM25** — the modern refinement of TF-IDF used by both Elasticsearch and Solr. Adds diminishing returns on term frequency and document length normalization.

Elasticsearch Architecture

- **Index** — analogous to a database; stores JSON documents
- **Shard** — each index is split into N shards, each a self-contained Lucene index; shards distribute across nodes for horizontal scaling
- **Replica shards** — copies of primary shards on different nodes for fault tolerance and read parallelism

```
Index: "safety_documents"
  Primary shard 0 → Node A | Replica 0 → Node B
  Primary shard 1 → Node B | Replica 1 → Node C
  Primary shard 2 → Node C | Replica 2 → Node A
```

Query DSL — Elasticsearch exposes a JSON query language with two fundamental types:

Full-text query (analyzed, relevance-scored):

```
{
  "query": {
    "match": { "content": "functional safety automotive" }
  }
}
```

Term query (exact match, not analyzed — for filters, enums, IDs):

```
{
  "query": {
    "term": { "status": "approved" }
  }
}
```

```
}  
}
```

Elasticsearch also doubles as an analytics engine — the foundation of the **ELK stack** (Elasticsearch + Logstash + Kibana) for log aggregation and monitoring.

Solr is functionally the same tool built on the same underlying library (Apache Lucene), with differences in operational model and configuration style. Elasticsearch has largely won mindshare for new projects.

—

Impact of LLMs on Full-Text Search

The introduction of LLMs shifted retrieval from **lexical** (token matching) to **semantic** (meaning matching). Documents are now retrieved by finding vectors nearest to the query vector — **vector search** or **ANN (Approximate Nearest Neighbor) search**. “Cardiac arrest” and “heart attack” now match because their embeddings are geometrically close, even with zero token overlap.

Hybrid search — running BM25 and vector search simultaneously and merging scores via **Reciprocal Rank Fusion (RRF)** — consistently outperforms either approach alone. BM25 remains strong for exact terms, rare vocabulary, and short queries; vector search handles semantic similarity.

RAG (Retrieval-Augmented Generation) is the pattern that made search infrastructure central to LLM applications. LLMs have a fixed context window and static knowledge cutoff — RAG solves this by retrieving only the most relevant chunks at query time:

```
User query  
↓  
Embed query → vector  
↓  
Search index (hybrid BM25 + vector)  
↓  
Top-K most relevant document chunks retrieved  
↓  
Chunks + query injected into LLM prompt  
↓  
LLM reasons over retrieved context → answer
```

Re-ranking adds a second precision pass after cheap retrieval:

```
Corpus (millions of docs)  
↓ BM25 + kNN (fast, cheap)  
100 candidates  
↓ Cross-encoder re-ranker (slower, more accurate)  
Top 5 chunks  
↓ LLM prompt  
Answer
```

For my FuSa project: the key architectural insight is the separation of the Retrieval & Ranking layer from the LLM reasoning layer. The LLM does not search — it reasons over what the retrieval layer surfaces. Quality of retrieval directly determines quality of LLM output.

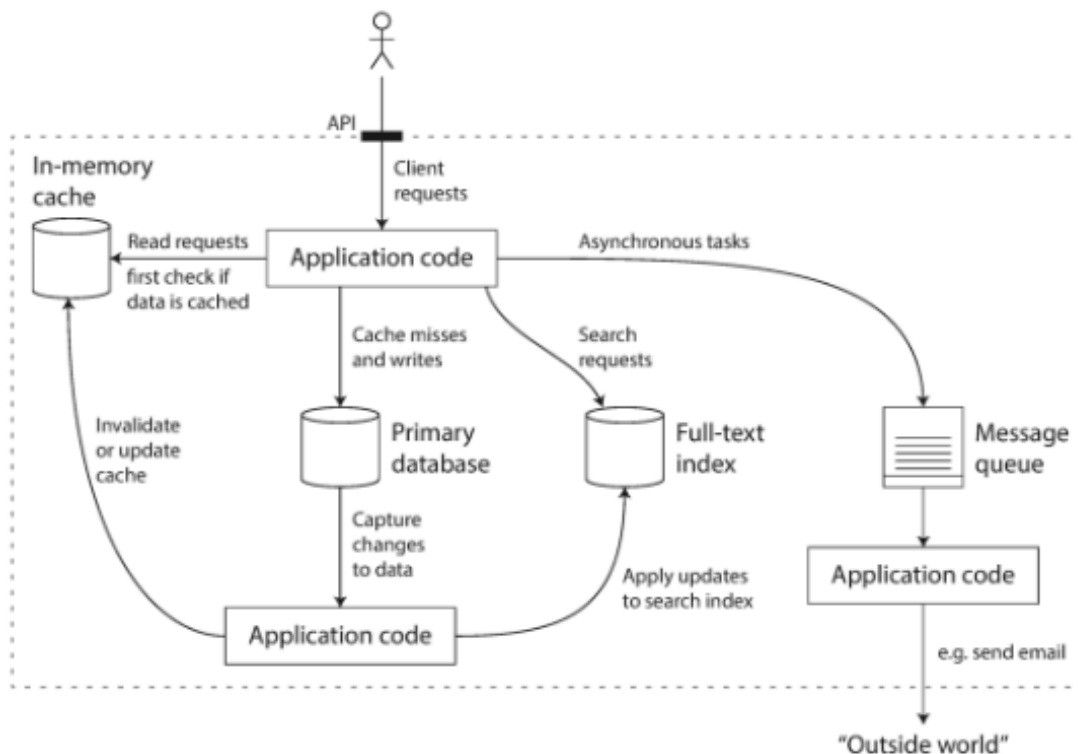


Figure 1: Sample Data System

1.1.3 Reliability

DDIA — Chapter 1, cont.

Reliability can be roughly described as “continuing to work correctly, even when things go wrong.”

- **Fault** — one component of the system deviating from its spec
- **Failure** — the system as a whole stops providing the required service to the user

Note: fault-tolerant systems are designed to prevent faults from causing failures — not to eliminate faults entirely.

Hardware Faults

Traditional response was hardware redundancy (RAID, dual power supplies, hot-swappable CPUs). Cloud platforms changed this: virtual machine instances can become unavailable without warning, so modern systems are increasingly designed to tolerate the loss of entire machines using software fault-tolerance techniques rather than relying on hardware redundancy alone.

Netflix Chaos Monkey — a program that deliberately triggers faults in production to test graceful error handling and validate fault-tolerance assumptions.

Software Errors

Systematic faults that tend to affect many instances simultaneously:

- A bug that crashes every application server instance when given a specific bad input
- A runaway process consuming shared resources — CPU, memory, disk, or network bandwidth
- A dependency that slows down, becomes unresponsive, or starts returning corrupted responses

Unlike hardware faults (which are largely random and independent), software errors are correlated — one bad input can take down the entire fleet at once.

Human Error

Configuration errors by operators are the leading cause of outages — more common than hardware failure or software bugs. Mitigations include: well-designed abstractions that make the right thing easy and the wrong thing hard, sandboxed staging environments, granular monitoring, and fast rollback paths.

—

1.1.4 Scalability

DDIA — Chapter 1, cont.

Term used to describe a system's ability to cope with increased load.

Describing Load.

Load parameters — the metrics used to describe load on a system:

- Requests per second to a web server
- The ratio of reads to writes in a database
- The number of simultaneously active users in a chat room
- The hit rate on a cache

Sometimes the average case matters; other times the bottleneck is dominated by a small number of extreme cases.

Case Study: Twitter

Operation	Average	Peak
Post Tweet	4.6k req/sec	12k req/sec
Home Timeline	300k req/sec	—

Twitter's scaling challenge is primarily due to **fan-out** — each user follows many people, and each user is followed by many people. Two approaches to implement this:

Approach 1 — Query on read: Posting a tweet inserts it into a global collection. When a user requests their home timeline, look up all the people they follow, find all tweets for each, and merge sorted by time.

Approach 2 — Cache per user (mailbox model): Maintain a home timeline cache for each user. When a tweet is posted, look up all followers and insert the tweet into each of their caches. Reading the home timeline is then cheap — the result is pre-computed.

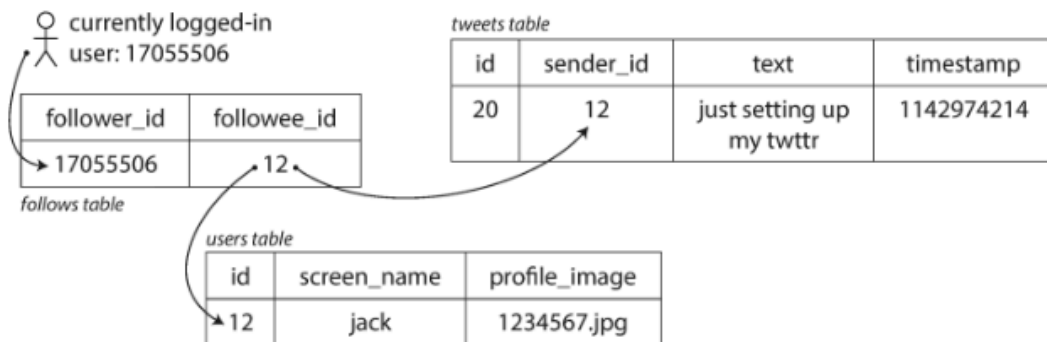


Figure 1-2. Simple relational schema for implementing a Twitter home timeline.

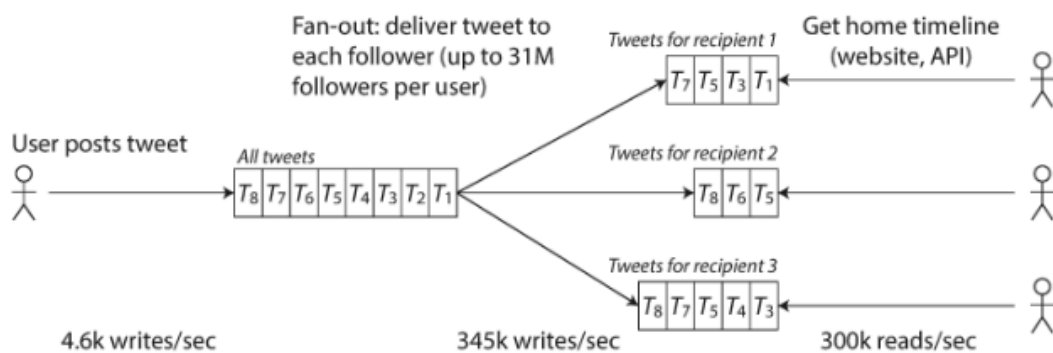


Figure 1-3. Twitter's data pipeline for delivering tweets to followers, with load parameters as of November 2012 [16].

Figure 2: Twitter Fan-Out Architecture

Approach 2 is elegant: using a per-user cache like a mailbox spreads the write load out across the system rather than concentrating it at read time. This kind of architectural thinking — moving work to where it's cheapest — is worth internalizing.

The tradeoff: when a high-follower account posts, the tweet must be written to millions of caches — a very expensive write. Twitter's solution is a hybrid: accounts above a follower threshold are handled with Approach 1; everyone else uses Approach 2. (*The real implementation is likely more sophisticated.*)

—

Describing Performance.

Two ways to think about the effect of increased load:

1. If you increase a load parameter and keep resources unchanged — how does performance degrade?
2. If you increase a load parameter — how much do you need to increase resources to maintain current performance?

Latency — the time a request spends waiting to be handled (in a “latent” state).

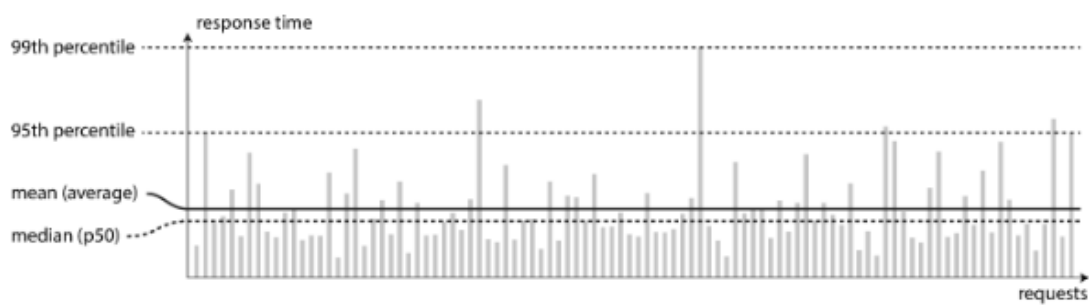


Figure 1-4. Illustrating mean and percentiles: response times for a sample of 100 requests to a service.

Figure 3: Response Time as a Distribution

Response Time — the total time for a response to reach the client. Includes latency, processing time, and network time.

Response time should be thought of as a **distribution**, not a single number — the same request will vary each time it runs.

Percentiles (Tail Latencies) — the standard way to reason about response time distributions. Common thresholds:

- **p90** — 90% of requests complete within this time
- **p99** — 99% of requests complete within this time
- **p999** — 99.9% of requests complete within this time

Percentiles are the basis for **SLOs (Service Level Objectives)** and **SLAs (Service Level Agreements)** — contracts defining expected performance and availability.

High percentiles are especially important in services where a single user request triggers multiple backend calls. Even with parallel calls, the user waits for the **slowest** of them to complete — slow tail latencies compound.

Calculating percentiles efficiently at scale is non-trivial. Approximate algorithms such as **forward decay**, **t-digest**, and **HdrHistogram** provide good accuracy at minimal CPU and memory cost.

Note: Averaging percentiles — to reduce time resolution or combine data from multiple machines — is mathematically meaningless. The correct way to aggregate response time data is to **add the histograms**.

—

Scaling Approaches.

Scale Up (Vertical Scaling) — move to a more powerful machine. Simple, but has a ceiling and can be expensive.

Scale Out (Horizontal Scaling) — distribute load across multiple smaller machines (also called a **shared-nothing architecture**). More complex, but no hard ceiling.

Most good architectures use a pragmatic mix of both.

Pragmatic wisdom: keeping a database on a single node is often simpler and the right default. In the modern cloud environment, when distribution becomes an engineering requirement, the tooling to do it without prohibitive upfront or maintenance cost is increasingly accessible.

1.1.5 Maintainability

Maintaining legacy systems is painful. Therefore we should design systems that are maintainable. There are three design principles to follow in this space.

Operability - Make it easy for the operations team to keep the system running smoothly.

Simplicity - Make it easy for new engineers to understand the system, by removing as much complexity as possible.

Evolvability - Make it easy for engineers to make changes to the system in the future, adapting it for unanticipated use cases as requirements change. Also known as extensibility, modifiability, or plasticity.

Operations teams are vital to keeping a software system running. A good operations team typically is responsible for the following, and more:

- Monitoring the health of the system and quickly restoring service if it goes into a bad state.
- Tracking down causes of problems
- Keeping software and platforms up to date, including security patches.
- Performing complex maintenance tasks, such as moving an application from one platform to another.

Things Data Systems can do to make Routine Tasks Easy:

- Providing visibility into the runtime behavior and internals of the system, good monitoring.
- Providing good support for automation and integration with standard tools.
- Avoiding dependency on individual machines
- Providing good default behavior, but also giving administrators the freedom to override the defaults when needed

Abstractions are very helpful in keeping code simple and maintainable, however finding good abstractions is very hard.

2 Part 2: Data models and Query Languages

2.1 Data Models

Data modeling is perhaps the single most important part of developing software. Most applications are built by layering one data model on top of another. For each layer, the key question is: how is it represented in terms of the next-lowest layer?

- 1.) As an application developer, you look at the real world (people, actions, money, sensors) and model it in terms of objects or data structures, and APIs that manipulate those data structures.
- 2.) When you want to store those data structures, you express them in terms of a general-purpose data model, such as JSON or XML documents, tables in a relational database, or a graph model.
- 3.) The engineers who built your database software decided on a way of representing JSON/XML/relational/graph data in terms of bytes in memory, on disk, or on a network. The representation may allow the data to be queried, searched, manipulated, and processed in various ways.
- 4.) On yet lower levels, hardware engineers have figured out how to represent bytes in terms of electrical currents, pulses of light, magnetic fields, and more.

The core idea is that each layer hides the complexity of the layers below it by providing a clean data model.

2.1.1 Relational Model Versus Document Model

The best known data model today is likely SQL, based on the relational model proposed by Edgar Codd in 1970.

Data is organized into relations (tables), where each relation is an unordered collection of tuples (rows).

The Birth of NoSQL.

In the 2010s, NoSQL was introduced, a non-relational database system driven by:

- A need for greater scalability than relational databases can easily achieve (very large datasets, or very high write through-put)
- A widespread preference for free and open source software over commercial database products
- Specialized query operations not well supported by relational models

The Object-Relational Mismatch.

Most applications are developed in object-oriented languages, which leads to a common criticism of the SQL data model: if data is stored in relational tables, an awkward translation layer is required between the objects in the application code and the database model of tables, rows, and columns.

ORM Frameworks help solve this problem (SQLAlchemy)

JSON datastructures are trees.

Many-to-One and Many-to-Many Relationships.

Standardized lists and auto-completers have clear advantages over plain text fields:

- Consistent style and spelling
- Ease of updating
- Localization support
- Better search functionality

IDs vs. Text Strings

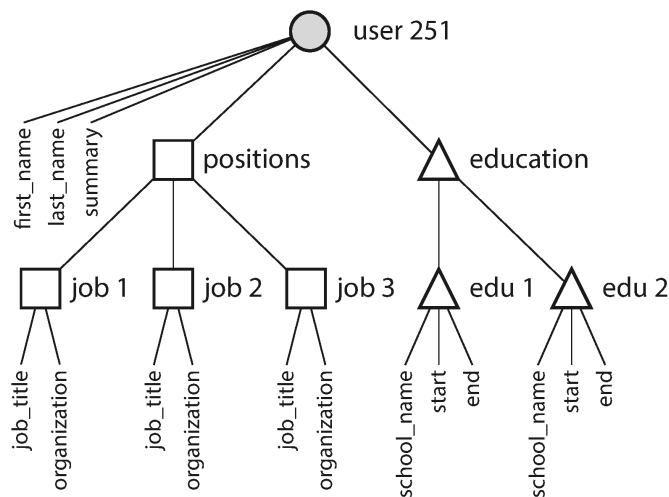


Figure 4: LinkedIn Profile JSON represented as a Tree.

When storing data, using an ID means the human-meaningful information (e.g. “Philanthropy”) lives in exactly one place — everything else references it by ID. Because the ID itself has no meaning to humans, it never needs to change even if the underlying information does. Text strings, by contrast, may need to change — and if duplicated, every copy must be updated, incurring write overhead and risk of inconsistency.

Removing this duplication is the key idea behind *normalization*.

The Document Model’s Join Problem

Normalized data requires *many-to-one* relationships, which don’t fit naturally into the document model. Relational databases handle joins natively; document databases don’t — so the application layer must compensate by making multiple queries.

Example — Recommendation Feature

A user can write a recommendation that appears on another user’s resume, showing the recommender’s name and photo. If the recommender later updates their photo, all recommendations they’ve written must reflect the change. Therefore the recommendation must store a *reference* to the recommender, not a copy of their data.

Hierarchical models and NoSQL work well for one-to-many relationships, but struggle with many-to-many.

—

Document Model vs. Relational Model

Document Model	Relational Model
Schema flexibility (schema-on-read)	Strong join support

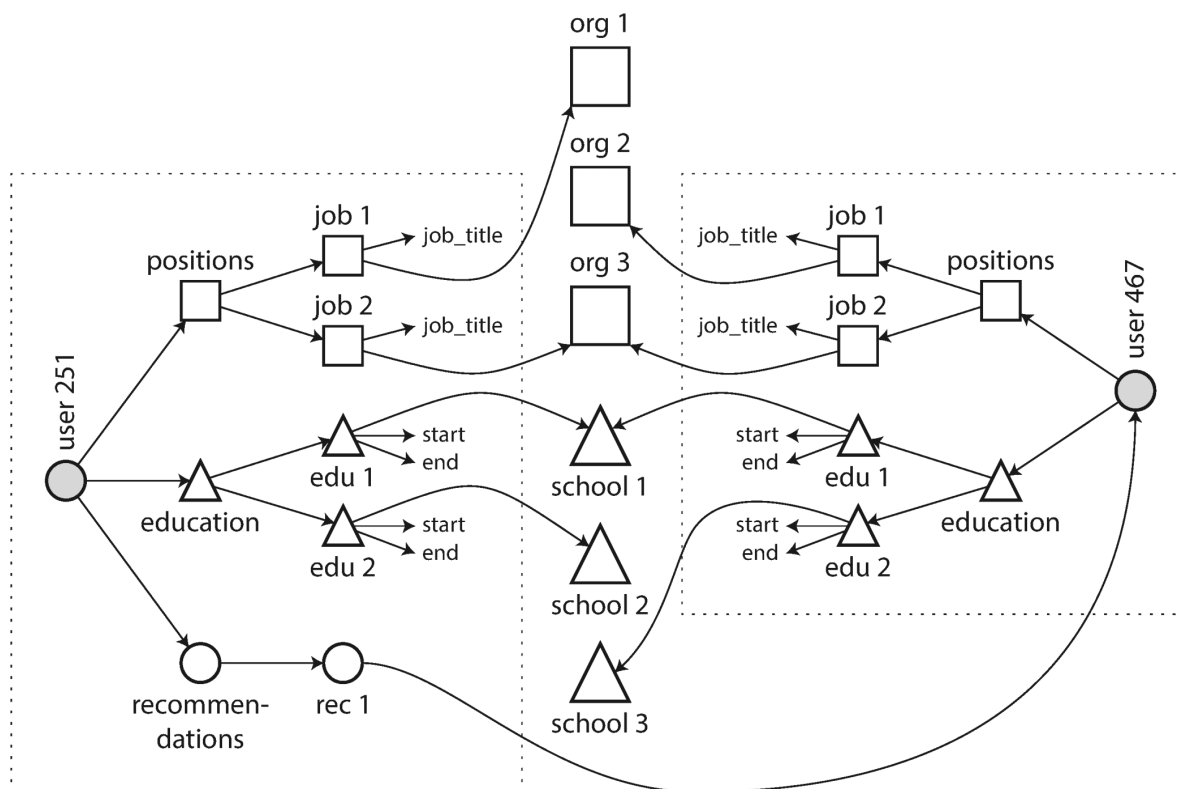


Figure 5: Many-to-Many Resume Graph

Better locality performance

Native many-to-one relationships

Closer to application data structures

Native many-to-many relationships

Struggles with joins

Schema-on-write (strict, consistent)

Schema-on-Read vs. Schema-on-Write

- **Schema-on-read** — structure is interpreted at read time. Analogous to dynamic (runtime) type checking in Python. Advantageous when items in a collection don't all share the same structure (heterogeneous data).
- **Schema-on-write** — structure is enforced at write time. Analogous to compile-time type checking.

Note: Data locality is not exclusive to the document model. Google's Spanner offers the same locality properties in a relational model by allowing a table's rows to be nested within a parent table. Most relational databases have also added XML/JSON support, and document databases are adding join solutions — the two models are converging.

Declarative vs. Imperative Query Languages

An **imperative** language tells the computer to perform specific operations in a specific order.

A **declarative** language lets you specify *what* you want — the conditions results must meet and how data should be transformed — without specifying *how* to achieve it. Declarative languages often lend themselves naturally to parallel execution.

```
-- Declarative example
SELECT * FROM animals WHERE family = 'Sharks';
```

MapReduce sits between these two paradigms. The query logic is expressed as code snippets (map/reduce functions from functional programming) that are called repeatedly by the processing framework — neither fully declarative nor fully imperative. Supported in limited form by MongoDB and CouchDB for read-only queries across many documents.

2.2 Graph-Like Data Models

If the application has mostly one-to-many relationships, or no relationships between records, the document model is appropriate. If many-to-many relationships are very common, and the data is complex, graph based models are the preferred approach.

Some typical examples include:

- **Social Graphs** — Vertices are people, edges indicate whether the people know each other.
- **The Web Graph** — Vertices are web pages, and edges indicate HTML links to other pages.
- **Road or Rail Networks** — Vertices are junctions, and edges represent the roads or railway lines between them.

Well-known algorithms can operate on these graphs: shortest path (Google Maps), PageRank (Google).

Graphs are also not limited to homogeneous data. Facebook maintains a graph with many different types of vertices and edges: vertices represent people, locations, events, and comments made by users; edges indicate which people are friends, who comments on what post, who attended what event, etc.

2.2.1 Graph query languages

Cypher (used by Neo4j) is the most widely adopted. It uses an ASCII-art syntax to express graph patterns — nodes are represented as `()` and relationships as `->` or `-[:TYPE]->`. For example:

```
MATCH (a:Person)-[:FRIENDS_WITH]->(b:Person)
WHERE a.name = 'Alice'
RETURN b.name
```

2.2.2 Property graphs

In the property graph model, each vertex consists of:

- A unique identifier
- A set of outgoing edges
- A set of incoming edges
- A collection of properties (key-value pairs)

Each edge consists of:

- A unique identifier
- The vertex at which the edge starts (the *tail* vertex)
- The vertex at which the edge ends (the *head* vertex)

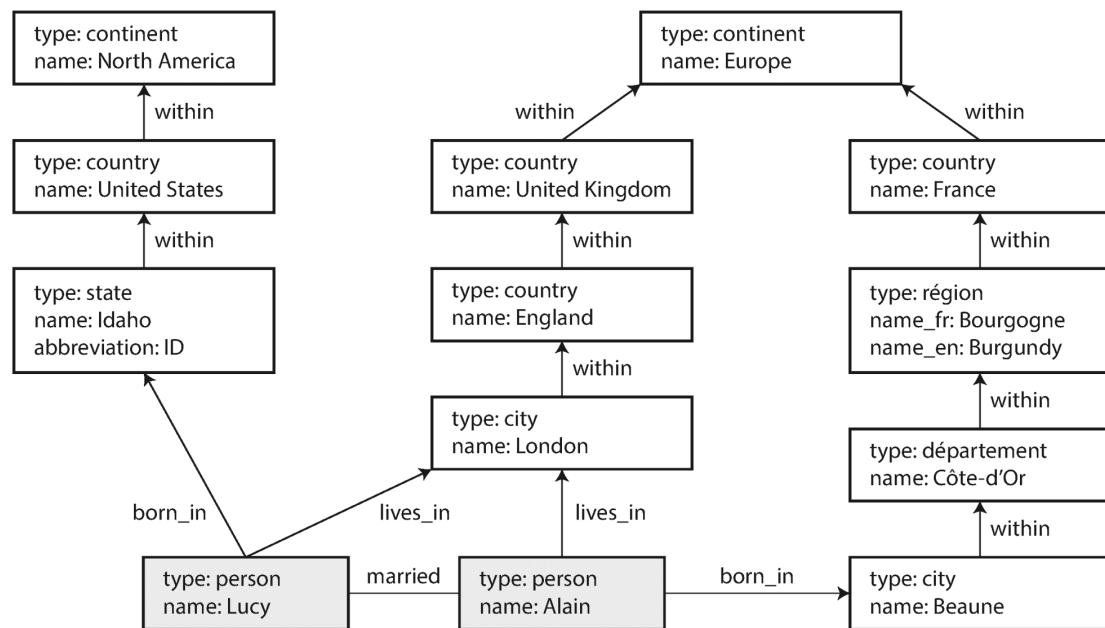


Figure 6: Edges and vertices with multiple types

- A label to describe the kind of relationship between two vertices
- A collection of properties (key-value pairs)

A Cypher query on this graph may look like:

```
MATCH (a:Person {name: "Alice"})-[:KNOWS]->(b:Person)
RETURN b.name, r.since
```

With graph databases we can begin to ask more interesting questions. For example: find the names of all people who emigrated from the United States to Europe — that is, all vertices with a BORN_IN edge to a location within the US, and a LIVES_IN edge to a location within Europe.

```
MATCH
  (person) -[:BORN_IN]-> () -[:WITHIN*0..]-> (us:Location {name: 'United States'}),
  (person) -[:LIVES_IN]-> () -[:WITHIN*0..]-> (eu:Location {name: 'Europe'})
RETURN person.name
```

This can be read as:

1. person has an outgoing BORN_IN edge to some vertex. From that vertex, follow a chain of outgoing WITHIN edges until reaching a Location vertex whose name is "United States".
2. That same person vertex also has an outgoing LIVES_IN edge. Following that edge, then a chain of WITHIN edges, eventually reaches a Location vertex whose name is "Europe".

This same query in SQL takes 29 lines of code — a clear illustration that different data models are designed to satisfy different use cases.



Figure 7: Graph data model example

2.2.3 Triple stores and SPARQL

Note: Triple stores and SPARQL are the right tool for a specific class of problems — heavily inter-connected, schema-heterogeneous, semantically rich data where interoperability with external ontologies matters. Outside that sweet spot, most teams find the tradeoffs unfavorable.

The triple store model is mostly equivalent to the property graph model. In a triple store, all information is stored in the form of simple three-part statements: *(subject, predicate, object)*. For example, in the triple *(Jim, likes, bananas)*, Jim is the subject, likes is the predicate, and bananas is the object.

The subject of a triple is equivalent to a vertex in a graph. The object is one of two things:

1. **A primitive value** such as a string or number. In this case, the predicate and object are equivalent to the key and value of a property on the subject vertex. For example, *(lucy, age, 33)* is like a vertex lucy with properties {"age": 33}.
2. **Another vertex in the graph.** In this case, the predicate is an edge, the subject is the tail vertex, and the object is the head vertex. For example, in *(lucy, marriedTo, alain)*, both lucy and alain are vertices, and marriedTo is the label of the edge connecting them.

Example data in Turtle format:

```
_:lucy      a :Person;    :name "Lucy";           :bornIn _:idaho.
_:idaho     a :Location;  :name "Idaho";           :type "state";       :within _:usa.
_:usa       a :Location;  :name "United States";    :type "country";     :within
_:namerica.
_:namerica a :Location;  :name "North America";    :type "continent".
```

2.2.4 The Semantic Web & Summary

Main Idea: Websites already publish information for humans to read, why don't they also publish the information in a way that is machine readable?

This idea has for the most part been abandoned in its original form and has been replaced with the vision of software agents powered by LLM's and RAG databases for enterprise and web retrieval.

Summary

There are three main database variations you should consider when building a system.

- 1.) **Document Databases** - Target use cases where data comes in self-contained documents and relationships between one document and another are rare.
- 2.) **Graph Databases** - Targeting use cases where anything is potentially related to everything.
- 3.) **Relational Databases** - Good for modeling many-to-many relationships.